分布式系统的调度算法

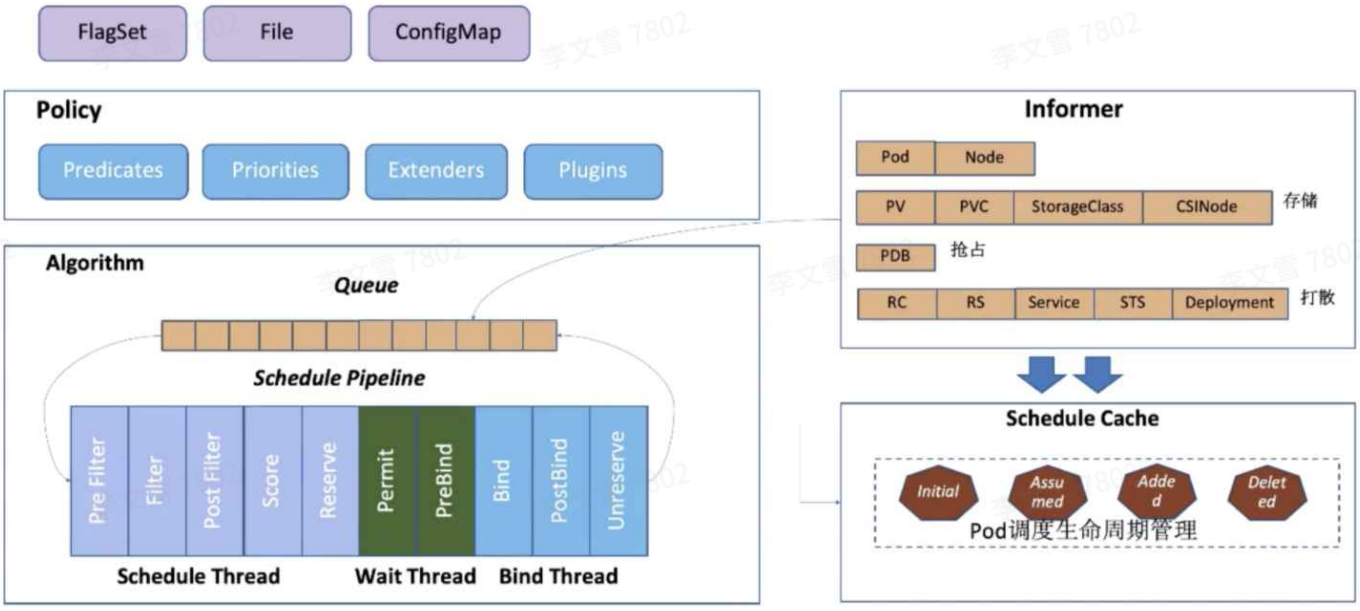
> author：李文雪

K8S default scheduler algorithm

Kubernetes （后简称K8S）是用于自动部署，扩展和管理容器化应用程序的开源系统。业界广泛使用K8S来管理公司的海量服务器。

尽管K8S的生态在日益成熟，K8S的架构设计却渐渐显示出不足，例如K8S的调度器 kube-scheduler。kube-scheduler存在许多问题，例如：可扩展性不好、调度算法过于简单、串行处理效率过慢等等。因此业界往往会对K8S做一些改造或是拓展。针对kube-scheduler的“可扩展性不好”、“串行处理效率过慢”等架构问题，业界已经有人在设计新的架构了。

下面以 **release-1.16** 为基础，介绍 kube-scheduler 。



调度流水线

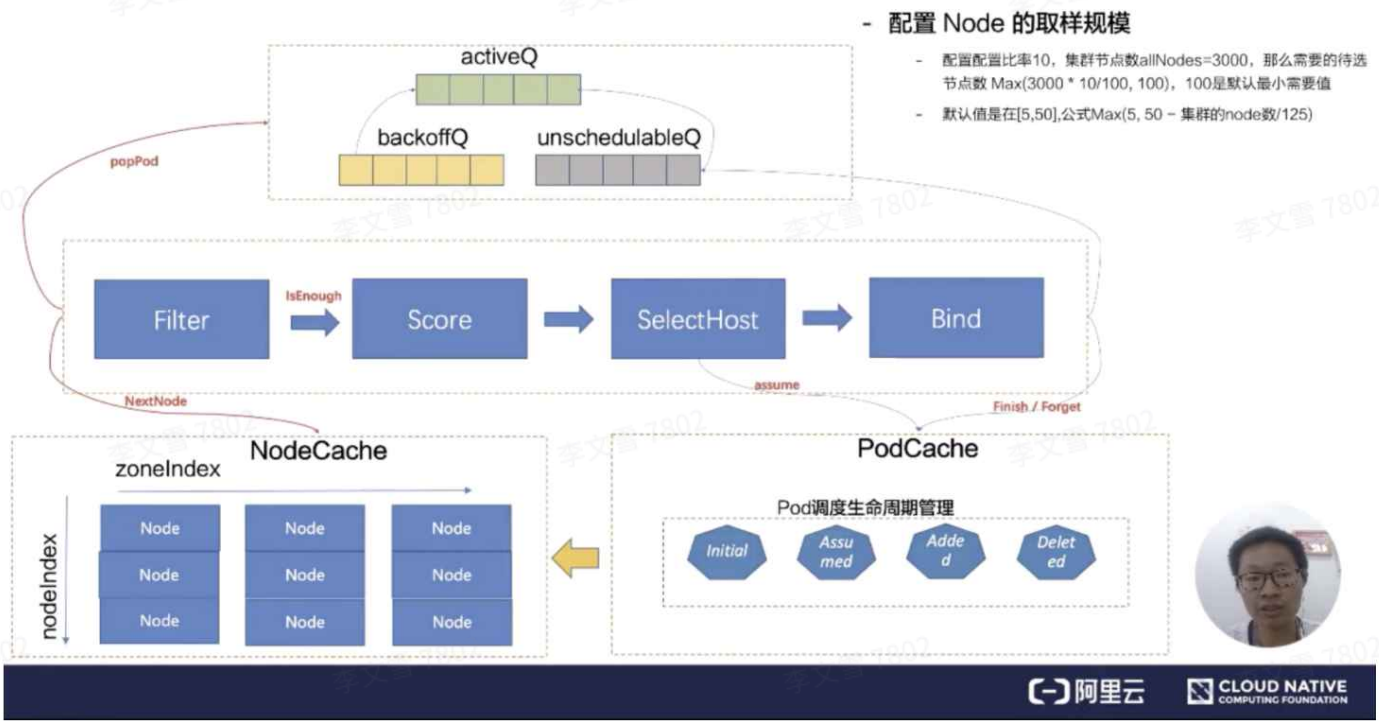
通过 **Informer** 将需要调度的 Pod 插入 Queue 中，Pipeline 会循环从 Queue Pop 等待调度的 Pod 放入 Pipeline 执行。

调度流水线 (Schedule Pipeline) 主要有三个阶段：Scheduler Thread，Wait Thread，Bind Thread。

- **Scheduler Thread 阶段**: 从如上的架构图可以看到 Schduler Thread 会经历 Pre Filter -> Filter -> Post Filter-> Score -> Reserve，可以简单理解为 Filter -> Score -> Reserve。
  - Filter 阶段用于选择符合 Pod Spec 描述的 Nodes；
  - Score 阶段用于从 Filter 过后的 Nodes 进行打分和排序；
  - Reserve 阶段将 Pod 添加到 排序后的最优 Node 的NodeCache 中，表示这个 Pod 已经分配到这个 Node 上, 让下一个等待调度的 Pod 对这个 Node 进行 Filter 和 Score 的时候能看到刚才分配的 Pod。
- **Wait Thread 阶段:** 这个阶段可以用来等待 Pod 关联的资源的 Ready 等待，例如等待 PVC 的 PV 创建成功，或者 Gang 调度中等待关联的 Pod 调度成功等等；
- **Bind Thread 阶段:** 用于将 Pod 和 Node 的关联持久化 Kube APIServer。

整个调度流水线只有在 Scheduler Thread 阶段是串行的一个 Pod 一个 Pod 的进行调度，在 Wait 和 Bind 阶段 Pod 都是异步并行执行。

调度详细流程（Scheduler Thread）



SchedulingQueue 有三个子队列：activeQ、backoffQ、unschedulableQ。

Scheduler 启动的时候所有等待被调度的 Pod 都会进入 activieQ，activeQ 会按照 Pod 的 priority 进行排序，Scheduler Pipepline 会从 activeQ 获取一个 Pod 进行 Pipeline 执行调度流程，当调度失败之后会直接根据情况选择进入 unschedulableQ 或者 backoffQ，如果在当前 Pod 调度期间 Node Cache、Pod Cache 等 Scheduler Cache 有变化就进入 backoffQ，否则进入 unschedulableQ。

unschedulableQ 会定期较长时间（例如 60 秒）刷入 activeQ 或者 backoffQ；backoffQ 会以 backoff 机制相比 unschedulableQ 比较快地让待调度的 Pod 进入 activeQ 进行重新调度。

1）Scheduler Thread

在 Scheduler Pipeline 拿到一个等待调度的 Pod，会从 NodeCache 里面拿到相关的 Node 执行 Filter 逻辑匹配，这从 NodeCache 遍历 Node 的过程有一个空间算法上的优化，简单可以概括为在避免过滤所有节点的同时考虑了搜索能力。

Score 阶段依据 Policy 配置的算分插件（Priority），进行排序，分数最高的节点作为 SelectHost。接着将这个 Pod 分配到这个 Node 上，这个过程叫做 **Reserved 也**可以称为账本预占。预占的过程修改 Pod 在 PodCache 的状态为 Assumed 的状态。

调度过程涉及到 **Pod 状态机的生命周期**，这里简单介绍下 Pod 的几个主要状态: **Initial**（虚拟状态）->**Assumed**（Reserved)->**Added**->**Deleted**(虚拟状态）; 当通过 Informer 看到 Pod 数据已经确定分配到这个节点的时候，才会把 Pod 的状态变成 Added。选中完节点在 Bind 的时候，有可能会 Bind 失败，在 Bind 失败的时候会做回退，就是把做 Assumed 的数据退回 Initial，也就是把 Assumed 状态擦除，从 Node 里面把 Pod 账本清除。

如果 Bind 失败，会把 Pod 重新丢回到 unschedulableQ 队列里面。如果在这么一个调度周期里面，Cache 发生了变化，会把 Pod 放到 backoffQ 里面。在 backoffQ 里面等待的时间会比在 unschedulableQ 里面时间更短，backoffQ 里有一个降级策略，是 2 的指数次幂降级。假设重试第一次为 1s，那第二次就是 2s，第三次就是 4s，第四次就是 8s，最大到 10s。

存在的问题：

- 打分器比较弱，只关注当前被调度的Pod，不关心大量Pod的整体分配，缺少统筹的能力。因此可以导致以下场景：一个集群里存在Node A和Node B，A、B各有两个CPU核，现来了一个请求3个CPU核的Pod，结果是这个Pod无法部署。
- 一旦确定了某个Pod和Node的关联关系，后期变不能更改，也就是无法主动移动Pod。

理想情况：

- scheduler可以主动的、渐进的将整体的Pod部署到最理想的状态，或是逼近理想部署状态。
- 要注意的是：scheduler不能在最开始获得所有Pod的信息，只能逐步获得。